	ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University)		Assignment 3
Branch	CSIT	Programme	**B.Tech**
Course Name	Full-Stack Web Development with MERN	Semester	6th
Course Code	CSE 3839	Academic Year	2025-2026
Lab Assignment: 3		**Topic: Designing a backend service**	

Learning Level (LL)	**L1**: Remembering	**L3:** Applying	**L5**: Evaluating
	L2: Understanding	**L4**: Analysing	**L6**: Creating

Q's	Questions	COs	LL
1.	Develop a MERN backend application for managing products in a **Shop** database. Create a backend folder Backend and perform the following tasks: a) Create a folder src/db and write a Node.js code snippet init.js that imports and calls the initDatabase() function to connect to a MongoDB database named Shop, and starts an Express server on port 3000. b) Analyze the initDatabase() function and explain the role of the following components in establishing the database connection: i) process.env.DATABASE_URL ii) mongoose.connect() iii) mongoose.connection.on('open')	**CO3**	**L3 L4**
2.	Design a Mongoose model (ProductSchema) for managing products in an online shop database. Each **product** should contain the following fields: ***name – name of the product (required, String)*** ***price – price of the product (Number)*** ***category – category of the product (String)*** ***stock – number of items available (Number)*** The schema should also automatically store the creation and update timestamps.	**CO3**	**L3 L4**
3.	After designing the product Model, the backend application needs a service layer to perform database operations. The service layer should interact with the product model and provide functions to manage product data. Students must create a service file named products.js inside the services folder. a) Write a function **createProduct({ name, price, category, stock })** in services/products.js that accepts name, price, category, and stock as	**CO3**	**L3,L4**

	input. b) Write service functions to retrieve products from the database **listAllProducts()** – returns all products and **listProductsByCategory(category)** – returns products belonging to a specific category c) Write service functions to update a product using its _id and delete a product using its _id : **updateProduct(productId, productData)** and **deleteProduct(productId)**		
4.	In the Shop backend application, the Product Service file (services/products.js) contains functions for managing product data such as creating, retrieving, updating, and deleting products. Define Jest test cases in products.test.js file to verify the correctness of these service functions. a) Write Jest test cases to verify the createProduct() function. The test should check: i) A product is successfully created when all required fields are provided. ii) The database assigns a valid _id to the product. iii) The createdAt and updatedAt timestamps are generated automatically. b) Write test cases for the following functions: i) listAllProducts() should return all products stored in the database. ii) listProductsByCategory(category) should return only products belonging to the specified category. c) Write test cases to verify: i) updateProduct(productId, data) correctly updates the product details. ii) deleteProduct(productId) removes the product from the database.	**CO3**	**L3, L4**
5	A Blog Post backend application uses Express routes to allow users to interact with blog posts stored in a MongoDB database. The routes communicate with the service layer (posts.js) which performs database operations. The folder must implement a routes file that defines HTTP endpoints for creating, retrieving, updating, and deleting blog posts. Write an Express GET route to retrieve blog posts from the database with	**CO3**	**L3,L4**

	Endpoint: **GET /api/v1/posts.** The route should accept optional query parameters such as author, tag, sortBy, sortOrder and return as such cases stated below: • All posts if no query parameter is provided • Posts filtered by author if author is given • Posts filtered by tag if tag is given • Return an error if both author and tag are provided. Implement the following routes Get a single post by ID:**GET api/v1/posts/:id.** The route should return: • The blog post if found • 404 Not Found if the post does not exist. Write routes to perform the following operations for update a blog post: **PATCH /api/v1/posts/:id .**This route should update the specified blog post.Similarly delete a blog post:**DELETE /api/v1/posts/:id.** The route should delete the post from the database as such : • Return 204 No Content if deletion is successful • Return 404 Not Found if the post does not exist.		
6	The application exposes RESTful APIs that allow clients to create, retrieve, update, and delete blog posts stored in a MongoDB database. Describe the purpose of the following HTTP methods used in REST APIs: GET,POST, PATCH and DELETE. Define an Express.js application used to handle API routes for a product post system. Explain the role of the following components in the Express application: • app.use(cors()) • app.use(bodyParser.json()) • postsRoutes(app)	**CO3**	**L3,L4**

1. Assignment carries a weightage of 20 marks out of 100
2. Only CO3 outcomes are covered.

Course Outcomes	CO1	Understand and configure a modern full-stack development environment using Node.js, MongoDB, Docker, and supporting tools.
	CO2	Design and implement RESTful backend services using Express, MongoDB (Mongoose), and unit testing frameworks.
	CO3	Develop interactive frontend applications using React and efficiently integrate them with backend APIs using modern data-fetching techniques.
	CO4	Apply DevOps practices to containerize, test, and deploy full-stack

		applications using Docker and CI/CD pipelines.
	CO5	Implement authentication, performance optimization, Search Engine Optimization techniques, and automated testing for scalable and secure web applications.
	CO6	Build and integrate advanced full-stack architectures using data aggregation, visualization, and GraphQL APIs with modern frontend clients.